

Lecture 10: Anti-Reversing Techniques

August 26, 2024

Overview

- ▶ Basic Approaches
- ▶ Eliminating Symbols
- ▶ Code Encryption
- ▶ Active Antidebugger Techniques
- ▶ Confusing Disassemblers
- ▶ Code Obfuscation
- ▶ Control Flow Transformations
- ▶ Data Transformations

Forward

- ▶ Cannot totally prevent reversing
- ▶ Can only wear reversers down
- ▶ Some techniques introduce various inefficiencies (speed, size, reliability)
- ▶ If you wish to protect your IP, you should apply anti-reversing
- ▶ Copy protections, in particular, need good anti-reversing
- ▶ Some platforms particularly vulnerable (.NET, Java) – a good obfuscator is recommended

Basic Approaches

Typically a combination is applied

Eliminating Symbolic Information Remove as much textual information as possible. Usually just stripping symbolic information. For bytecode platforms, symbols must be renamed, but preserve linkage.

Obfuscating the Program Specifically aimed at preventing *static analysis*. Rearrange program elements and layout, but preserve its function.

Embedded Antidebugger Code Specifically aimed at preventing *dynamic analysis*. Program attempts to detect, damage, and/or disable any attached debugger(s). May simply detect the debugger and terminate, or interfere with it.

- These tend to be platform or even debugger specific.

Eliminating Symbolic Information

- ▶ For hardware platforms, it's almost automatic:
 - ▶ Symbols are not typically included in a release build, but be sure to inspect the program before it ships.
 - ▶ There are some symbols in import and export tables, though, and they can be revealing, esp. with C++, which includes class names.
 - ▶ May be worthwhile to import/export by ordinals instead, or obfuscate names.
- ▶ Different with bytecode platforms, since symbols are used to resolve internal references.
 - ▶ Cannot simply remove the symbols – they must be replaced instead.
 - ▶ Typically rename symbols after program is compiled.
 - ▶ May, e.g., use a mixture of 'l', 'I', and '1'.

Code Encryption

- ⊕ Very common to prevent static analysis.
 - ▶ Part or whole encrypted after compilation.
 - ▶ Decryption code must be embedded inside the executable.
- ⊖ Usually just an inconvenience – everything needed to decrypt it (logic and key) is contained in the program.
- ⊖ A decrypted copy (whole or part) must exist somewhere during runtime.
- ⊕ Most reversers will have to do some dynamic analysis to recover the original binary.

Code Encryption

- An unpacker will take care of most common encryption and packing techniques (e.g., UPX).
- ▶ Suggestion: Use a common decryption logic, but hide each key for each part of the program in a unique way.
- ▶ Consider generating each key dynamically via computations involving global variables spread about the program.

Beware The OS may corrupt encrypted code, if it attempts any relocation fixups.

Active Antidebugger Techniques

There are some techniques for older debuggers in the book. Note that debuggers have advanced since 2005, nevertheless, these techniques are still fairly effective.

- ▶ Seeking to detect a debugger or to hinder debuggers from performing their usual functions.
- ▶ Particularly effective with code encryption (hinder static and dynamic analysis).
- Some techniques cause false positives, causing, e.g., the program to terminate unpredictably under normal use (reliability hit)

More at <http://www.symantec.com/connect/articles/windows-anti-debug-reference>

Active Antidebugger Techniques

Debugger Basics

- ▶ Recall *software* breakpoints – implemented by writing an INT 3 instruction over the memory at the breakpoint.
 - ▶ Original instruction is replaced when breakpoint is removed.
- ▶ Alternative is to use *hardware* breakpoints – implemented in hardware registers; no modification to program in memory.
 - ▶ May also break on data access.
- ▶ Single-stepping is implemented on x86 via the *trap flag* in EFLAGS register.
 - ▶ When set, the processor generates interrupt number 1 after every instruction.

Active Antidebugger Techniques

IsDebuggerPresent

- ▶ Can trivially detect a user-mode debugger (e.g., OllyDbg, WinDbg)
 - ▶ Reads the current process's *Process Environment Block* (PEB)
- ▶ Also trivial for a reverse to detect and bypass
- ▶ Alternatively, you can access the PEB directly, avoiding an obvious call to IsDebuggerPresent

```
mov     eax, fs:[00000018]
mov     eax, [eax+0x30]
cmp     byte ptr [eax+0x2], 0
je      RunProgram
; Otherwise terminate
```

Active Antidebugger Techniques

IsDebuggerPresent

- ▶ You should probably terminate in some unobtrusive way, so the protection code is difficult to locate easily.
- Makes assumption that `IsDebuggerPresent`'s implementation will not change in future releases of Windows.
 - ▶ Recall `fs:[00000018]` contains the address of the *Thread Environment Block* (TEB).
 - ▶ The first **mov** reads the address of the PEB from offset `0x30` of the TEB.
 - ▶ The second **mov** reads offset `0x2` from the PEB, which is a byte indicating that a debugger is attached.
 - ▶ Any change to these internal structures may cause a crash or a false positive.

Active Antidebugger Techniques

SystemKernelDebuggerInformation

- ▶ `ZwQuerySystemInformation(`
 `SystemKernelDebuggerInformation ,`
 `(PVOID) &DebuggerInfo ,`
 `sizeof ( DebuggerInfo ) ,`
 `&ulReturnedLength ) ;`
- ▶ **`typedef struct`**
 `_SYSTEM_KERNEL_DEBUGGER_INFORMATION {`
 `BOOLEAN DebuggerEnabled ;`
 `BOOLEAN DebuggerNotPresent ;`
 `} SYSTEM_KERNEL_DEBUGGER_INFORMATION ,`
 `*PSYSTEM_KERNEL_DEBUGGER_INFORMATION ;`

— Will not catch SoftICE (defunct, anyway)

Active Antidebugger Techniques

“\\.\SIWVID”

- ▶ You can detect if SoftICE is *installed* by trying to open “\\.\SWVID”
- The mere installation of a kernel debugger does not imply one is trying to cheat you.
- ▶ In general, be wary of debugger-specific techniques as you require several of them to cover a variety of debuggers.
- ▶ Each technique you apply degrades the quality of the software and increases the probability of false positives.

Active Antidebugger Techniques

int 1

- Aimed specifically at SoftICE
 - ▶ SoftICE installs a handler for int 1 for single stepping
 - ▶ You can assemble one and see if anything other than the usual Access Violation occurs

```
▶ __try {  
    __asm int 1;  
}  
__except (TestSingleStepExec (GetExceptionInformation ())) {  
}
```

```
int TestSingleStepExec (  
    LPEXCEPTION_POINTERS pExceptionInfo) {  
    DWORD ExceptionCode =  
        pExceptionInfo->ExceptionRecord->ExceptionCode;  
    if (ExceptionCode != STATUS_ACCESS_VIOLATION)  
        printf ("SoftICE is present!\n");  
  
    return EXCEPTION_EXECUTE_HANDLER;  
}
```

Active Antidebugger Techniques

The Trap Flag

- ⊕ Works on any debugger (user or kernel)
 - ▶ Set the trap flag and see if an exception is “swallowed”

```
▶ BOOL bExceptionHit = FALSE;
__try {
    __asm {
        pushfd
        or dword ptr [esp], 0x100 // "Or in" the Trap Flag
        popfd
        nop
    }
}
__except(EXCEPTION_EXECUTE_HANDLER) {
    bExceptionHit = TRUE; // Good. no debugger.
}

if (bExceptionHit == FALSE)
    printf("A debugger is present!\n");
```

- ⊖ PUSH/POPCD stand out
- ⊖ Only detects tracing

Active Antidebugger Techniques

Code Checksums

- ▶ Compute a checksum on all or sensitive parts of the program
- ▶ Compare checksums computed at runtime with pre-computed values stored in the program
- ⊕ Effective against debuggers and code patching
- ⊖ Can be expensive (will slow down legitimate users)
 - ▶ Best to pick small sensitive functions called at program start up (surprisingly common)
 - ▶ Create intentional, inconspicuous rabbit holes that lead to termination
- ⊖ Not effective against hardware breakpoints

Confusing Disassemblers

- Not particularly robust
 - ▶ x86(_64) use variable-length instructions – can de-synchronize disassembler
 - ▶ Recall: linear sweep vs. recursive traversal
 - ▶ Recursive traversal disassemblers are more robust against these tricks

Disassembler/Debugger	Disassembly Method
OllyDbg	Recursive traversal
NuMega SoftICE	Linear sweep
Microsoft WinDbg	Linear sweep
IDA Pro	Recursive traversal
PEBrowse	Recursive traversal

Confusing Disassemblers

Linear Sweep Disassemblers

```
_asm {  
    // Some code ...  
    jmp After  
    _emit 0x0f  
After:  
    mov eax, [SomeVariable]  
    push eax  
    call AFunction  
}
```

↓ (OllyDbg – Recursive traversal – Correct)

0040101D	EB 01	JMP SHORT disasmtest.00401020
0040101F	0F	DB 0F
00401020	8B45 FC	MOV EAX,DWORD PTR SS:[EBP-4]
00401023	50	PUSH EAX
00401024	E8 D7FFFFFF	CALL disasmtest.401000

Confusing Disassemblers

Linear Sweep Disassemblers

↓ (SoftICE – Linear Sweep – Broken)

```
001B:0040101D  JMP      00401020
001B:0040101F  JNP      E8910C6A
001B:00401025  XLAT
001B:00401026  INVALID
001B:00401028  JMP      FAR [EAX-24]
001B:0040102B  PUSHAD
001B:0040102C  INC      EAX
```

(Step over the JMP)

```
001B:0040101D  JMP      00401020
001B:0040101F  JNP      E8910C6A
001B:00401020  MOV      EAX, [EBP-04]
001B:00401023  PUSH     EAX
001B:00401024  CALL     00401000
```

Confusing Disassemblers

Recursive Traversal Disassemblers

- ▶ Because the recursive traversal disassemblers trace jumps, you'll need to convince it to disassemble the junk.
- ▶ However, you can never allow the junk to actually execute, or the program will crash.
- ▶ This is accomplished with *opaque predicates*, which are always True or always False, but are difficult to fold/simplify.
- ▶

```
if (1 == 2) {  
    // Junk  
} else {  
    // Real code  
}
```

Confusing Disassemblers

Recursive Traversal Disassemblers

```
_asm {  
    mov eax, 2  
    cmp eax, 2  
    je After  
    _emit 0xf  
After:  
    mov eax, [SomeVariable]  
    push eax  
    call AFunction  
}
```

↓ (IDA – Recursive Traversal – Broken)

```
.text:00401031      mov     eax, 2  
.text:00401036      cmp     eax, 2  
.text:00401039      jz      short near ptr loc_40103B+1  
.text:0040103B loc_40103B: ; CODE XREF: .text:00401039 _j  
.text:0040103B      jnp     near ptr 0E8910886h  
.text:00401046      fsub    qword ptr [eax+40h]  
.text:00401049      add     al, ch  
.text:0040104B      add     eax, [eax]
```

Confusing Disassemblers

Recursive Traversal Disassemblers

- ▶ Some disassemblers prefer the true path (the jump target)
- ▶ Some disassemblers may have code to deal specifically with these tricks
- ▶ PEBrowse actually interprets both paths, showing overlaid instructions

```
#define paste(a, b) a##b
#define pastesymbols(a, b) paste(a, b)
#define OBFUSCATE() \
_asm { mov    eax, __LINE__ * 0x635186f1          }; \
_asm { cmp    eax, __LINE__ * 0x9cb16d48          }; \
_asm { je     pastesymbols(Junk, __LINE__)         }; \
_asm { mov    eax, pastesymbols(After, __LINE__)   }; \
_asm { jmp    eax                                   }; \
_asm { pastesymbols(Junk, __LINE__):                }; \
_asm { _emit (0xd8 + __LINE__ % 8)                 }; \
_asm { pastesymbols(After, __LINE__):                }; \
```

Confusing Disassemblers

Recursive Traversal Disassemblers

- ▶ This particular technique is fairly easy to defeat
- ▶ Powerful disassemblers allow a user to provide hints to overcome them
- ▶ A script can detect and defeat most or all of them
- ▶ They do little except to annoy a reverser
- ▶ Would be better to use a Data- and Control-flow obfuscations (next)

Code Obfuscation

- ▶ Goal is to make code less human readable
- ▶ *potency* – the level of complexity added by obfuscation
- ▶ Idea is to bury the meaningful code in a sea of useless code
- ▶ Potency can be measured – predicate count, nesting depth, etc.
- ▶ *resilience* – difficulty of undoing an obfuscation transformation
- ▶ *deobfuscator* – tries to undo obfuscation
- ▶ Some transformations don't introduce a run-time cost (just reorganize code)
- ▶ Most usually do incur some cost (larger, slower, etc.)
- ▶ Automatic obfuscators are preferable to manual (maintains readable source)

Control Flow Transformations

- ▶ Computation Transforms¹
 - ▶ Adding and removing control-flow elements that preserve the computation result
- ▶ Aggregation Transforms
 - ▶ Breaks high-level abstractions created by the programmer
 - ▶ Makes high-level organization senseless – does not reflect purpose
- ▶ Ordering Transforms
 - ▶ Change order of operations, again preserving computation result

¹[Colberg1] in Textbook bibliography

Control Flow Transformations

Opaque Predicates

- ▶ Examples given before, e.g., $(x + 1 == x)$, are too trivial
- ▶ Need predicates that:
 - ▶ are difficult to distinguish from program code
 - ▶ have a constant known value
 - ▶ cannot have their value determined easily without stepping through code
- ▶ Confusing a reverser is different than confusing a tool

Control Flow Transformations

Opaque Predicates

- ▶ e.g., multiple threads modifying a global according to a rule
 - ▶ Predicates know the rule
 - ▶ Complicates matter for deobfuscators
 - ▶ Will look fishy (stand out) to a reverser
- ▶ Can also add bogus members into data structures
 - ▶ embed code to modify them according to a numeric rule
- ▶ In general, for low-level languages, focus on confusing a human reverser; for bytecode languages, focus on confusing tools

Control Flow Transformations

Confusing Decompilers

- ▶ Especially important for bytecode-base languages
- ▶ Goal is to prevent decompiler from reconstructing the control-flow structure
- ▶ Consider Java's goto bytecode instruction
- ▶ Certain arrangements have to corresponding high-level equivalent
- ▶ Can transform program in a way that uses these instructions as such
- ▶ Native decompilation is complex and fragile
- ▶ Almost any form of obfuscation will cause them to fail

Control Flow Transformations

Table Interpretation

- ▶ Can break automated deobfuscators and confuse human reversers if done well
- ▶ Idea is to break a function down into chunks and order them via a table
- ▶ Similar to a state machine, where state positions in table are randomized
- ▶ Essentially a switch statement nested in a loop
- ▶ Can also be thought of as a little virtual machine, where the table is the opcode sequence
- ▶ Especially effective when combined with a disassembly obfuscation technique – prevents a global view of the table and operations

Control Flow Transformations

Table Interpretation

- ▶ There are lots of possible enhancements
- ▶ e.g., use an additional table, computed at runtime, to add more indirection and hinder static analysis
- ▶ e.g., use pointer aliases to create such a table at runtime

Performance

- ▶ Author's results: An XOR loop with a runtime-computed indirect table interpretation ran 3.8 times slower
- ▶ Applying additional OBFUSCATES increased it to about 12!
- ▶ Must consider tradeoffs of protection vs. performance
- ▶ Only sensitive parts of program need protection

Control Flow Transformations

Inlining and Outlining

inlining optimizing by copying a function where it would be called

- ▶ Also removes some internal abstractions used by the developer

outlining moving a code sequence from within a function into its own function

- ▶ Done repeatedly can add to the “confusion factor”

Control Flow Transformations

Interleaving Code

- ▶ Interleave the code of several functions into the same area of memory
- ▶ Break each into segments, shuffle the segments together, and use opaque predicates to glue them back
- ▶ More effective when predicates are difficult to distinguish from function code
- ▶ More effective when the functions are related and/or perform similar computations
- Quite costly in size and speed
- ▶ Can also merge the entry point and use a parameter to select which original function

Control Flow Transformations

Ordering Transformations

- ▶ Randomize the order of operations
- ➖ Requires parts of the function to have non-codependent operations
- ▶ Recommended for automated obfuscators, since manual obfuscation of operations can be very confusing to developers
- ➕ Cheap, and decently effective against humans

Data Transformations

- ▶ Discovering data structures is an important part of reversing a program
- ▶ So make this key step more difficult
- ▶ Of course, transforming data structures requires changing the code that interacts with them

Data Transformations

Modifying Variable Encoding

- ⊕ Delays a reverser's intuition when determining the purpose of a variable
 - ▶ e.g., simply shift the value one bit to the left
 - ▶ e.g., having loop counters step by 2
- ⊖ Compilers have a tendency to remove this sort of thing
 - ▶ These examples are way too trivial, but the concept can be expanded
 - ▶ Consider homeomorphisms (graph theory), though this can be expensive

Data Transformations

Restructuring Arrays

- ▶ Rearranging one or more arrays such that function is preserved, but becomes confusing to reversers
- ▶ e.g., merging one or more arrays by interleaving elements
- ▶ e.g., break one array down into more than one
- ▶ e.g., change the number of dimensions
- Fairly easy to reconstruct

Conclusion

- ▶ Usually just slowing down reversers
- ▶ Antidebugger tricks and code obfuscation are most common anti-RE tricks
- ▶ Antidebugger tricks
 - ▶ Prevent a reverser from using a debugger, or confusing it
 - ▶ Tend to be particular to a debugger
- ▶ Code obfuscation

Theme Rearrange components, preserve function, confuse tools and humans

- ▶ Almost always incur a significant runtime penalty
- ▶ Must balance piracy concerns with performance (economics)